

Frequent Itemset Mining and Scalability Analysis on the IMDb Top 1000 Dataset

Diego Alonso Leiva Cordova (ID: 58757A)

Data Science for Economics (DSE)

Algorithms for massive data

Università degli Studi di Milano

Milan, Italy

March 7, 2026

Contents

1. Introduction	3
2. General information	4
2.1. Data	4
2.2. Analysis and Preprocessing	4
3. Methodology and algorithms	4
3.1. A-priori algorithm	5
3.1.1. Main memory limitation	7
3.1.2. Multi-Actor Item sets	7
3.1.3. Association Rules	8
3.2. Alternative models	9
3.2.1. SON algorithm	10
3.2.2. Toivonen's algorithm	11
4. Experiments	13
4.1. Different supports	13
4.2. Different sizes	14
5. Scalability	15
5.1. First scalability	15
5.2. Second scalability	16
5.3. Third scalability	16
6. Conclusion	17
7. References	18

1. Introduction

The following report describes the application of frequent itemset mining techniques to analyse relationships between actors, memory limits, and algorithm efficiency in the IMDb Top 1000 dataset.

As mentioned in a book *“The market-basket model of data is used to describe a common form of many-many relationship between two kinds of objects. On the one hand, we have items, and on the other we have baskets, sometimes called “transactions.” Each basket consists of a set of items (an itemset), and usually we assume that the number of items in a basket is small– much smaller than the total number of items. The number of baskets is usually assumed to be very large, bigger than what can fit in main memory.”* (Jure Leskovec, 2014)

Each film is modelled under the market-basket model, where a movie represents a basket and the actors listed in the fields Star1, Star2, Star3 and Star4 represent the items within that basket, so after performing basic data preprocessing, the dataset is transformed into a transactional format suitable for pattern mining.

The A-priori algorithm is first applied to identify frequent actor combinations and to generate association rules that reveal meaningful patterns between actors in films. Then, SON algorithm and Toivonen’s algorithm, designed to handle larger datasets and lower support thresholds, are implemented. Two experiments are conducted to evaluate the behaviour of these algorithms under different support thresholds, and dataset sizes.

Finally, scalability is performed to analyse how the algorithms perform as the dataset grows, highlighting the trade-offs between the classical frequent pattern mining method A-priori, with SON and Toivonen’s algorithms.

2. General information

2.1. Data

The project relies on the dataset IMDB Movies Dataset-Top 1000 Movies by IMDB Rating published in Kaggle which is an online platform with a huge repository of community-published models, data and code. (Shankhdhar)

2.2. Analysis and Preprocessing

The dataset contains 1000 rows (1000 films) and 16 columns, where the column 'Series_Title' refers to the film's name. All films and actors are completed. There are no duplicates rows and to uniformize the actor's names, those are put without extra space and in lowercase.

3. Methodology and algorithms

This section is the central part of the project that provides a brief explanation of the theory, the models used, and evaluations of their capabilities and limitations.

Before passing the classic A-priori algorithm, consider these concepts:

- * Itemset Size (k): Are the number of items contained within a specific candidate or frequent set during a given iteration. For this project, k defines the number of actors being analysed in a combination, where the algorithm progresses from individual actors ($k=1$) to pairs ($k=2$), triples ($k=3$), until $k=4$.
- * Support Threshold (s): Is the minimum frequency that an itemset must reach to be considered significant and "frequent". In this context, s determines the minimum number of films in which a group of actors must appear together for their relationship to be identified as a pattern.

3.1. A-priori algorithm

“In the A-Priori Algorithm, one pass is taken for each set-size k . If no frequent itemsets of a certain size are found, then monotonicity tells us there can be no larger frequent itemsets, so we can stop. The pattern of moving from one size k to the next size $k + 1$ can be summarized as follows. For each size k , there are two sets of itemsets: C_k is the set of candidate itemsets of size k — the itemsets that we must count to determine whether they are in fact frequent. L_k is the set of truly frequent itemsets of size k .” (Jure Leskovec, 2014)

In practice, the “mlxtend” library runs the full A-Priori algorithm on python. This algorithm operates through iterative cycles (multiple passes):

- * Pass 1: Finds frequent individual items (L_1) who appear in more than s films.
- * Pass 2: Combines items from the previous step to find frequent pairs (L_2).
- * Pass 3: Takes the frequent pairs from step 2 and combines them to try to form candidate triplets (C_3). If those triplets exceed the minimum support, they become frequent triplets (L_3).
- * Pass 4: Generates candidate quadruplets (C_4). This is the maximum limit. The algorithm will attempt to see whether there are 4 actors who always appear together in the 4 columns in a film.

End of the Algorithm: The algorithm stops because there is no basket with 5 actors, so the support of any quintuplet would automatically be 0.

According to the lecture *“After the first pass, we examine the counts of the items to determine which of them are frequent as singletons. It might appear surprising that many singletons are not frequent. But remember that we set the threshold s sufficiently high that we do not get too many frequent sets; a typical s would be 1% of the baskets.” (Jure Leskovec, 2014).*

The first support threshold is set in 0.01, getting the following result:

Table 1. Most frequent actors of the dataset ordered by support

	support	itemsets
7	0.017	(robert de niro)
8	0.014	(tom hanks)
0	0.013	(al pacino)
1	0.012	(brad pitt)
3	0.012	(clint eastwood)
6	0.011	(matt damon)
2	0.011	(christian bale)
5	0.011	(leonardo dicaprio)
4	0.010	(james stewart)

The maximum s is 0.017. Robert De Niro appears in 1.7% of films, meaning that appears in 17 of the 1000 films. The same logic applies to the rest of actors.

Then, is testing different support threshold to see the number of item sets.

Table 2. Number of item sets at different thresholds

	support	num_itemsets
0	0.0170	1
1	0.0050	83
2	0.0020	791
3	0.0012	791

As expected, only one itemset (De Niro) has s 0.017, as s decreases the number of item sets increases, where 0.0020 and 0.0012 have the same number since 1.2 is equivalent to 2 films, because the algorithm works with whole numbers. From now on 0.002 is considered the lowest support with 791 item sets.

3.1.1. Main memory limitation

When the support threshold was lowered to 0.001, the algorithm failed due to memory limitations with this message: "The session crashed because all available RAM was used". With $s = 0.001$, the algorithm returns all item sets that appear in at least 1 film and that involves practically every possible combination of actors.

To find the frequent pairs, the algorithm follows these steps according to the theory:

- Step 1: Identify frequent items (L1) First, it finds which actors appear in more than s films. At $s = 0.001$, all 2,709 actors are considered frequent.
- Step 2: Generate Candidates (C2) This is where the memory failure could occur. The algorithm generates all possible combinations of the frequent actors to create a data structure where it will accumulate the counts. Since the actors are represented as 2709 columns, A-Priori generates the combination of every column with one another. This means $\binom{2709}{2} = 3,667,986$ candidate pairs. For $k=3$, $k=4$ the candidate pairs are even higher.

3.1.2. Multi-Actor Item sets

The next table shows combinations of two or more actors ($k \geq 2$), reducing the support threshold to 0.002 (it's enough that they've appeared together in 2 films).

Table 3. Combinations of actors with $k \geq 2$ and $s = 0.002$

	support	itemsets	length
689	0.006	(rupert grint, daniel radcliffe)	2
687	0.005	(emma watson, daniel radcliffe)	2
774	0.005	(emma watson, daniel radcliffe, rupert grint)	3
701	0.005	(emma watson, rupert grint)	2
723	0.004	(robert de niro, joe pesci)	2

The maximum support is 0.006, corresponding to Daniel Radcliffe and Rupert Grind, it means that they appeared together in 6 films. It is also present a length 3 with

support 0.005 to the same previous actors and Emma Watson. This trio happened automatically because in Pass 2, the algorithm noticed that:

- Daniel and Rupert are a frequent pair (6 movies).
- Daniel and Emma are a frequent pair (5 movies).
- Rupert and Emma are a frequent pair (5 movies).

Since all subsets were frequent, A-Priori generated the 3-actor candidate in its third pass and confirmed that they do, in fact, exceed the 0.002 threshold.

3.1.3. Association Rules

Until now the central concept is the Support threshold (s) that is applied over item sets (e.g. {Robert De Niro, Joe Pesci}), and answers the question: How common is this group? The next central concept is Confidence (c) that is applied over implications (e.g. Robert De Niro $\rightarrow$ Joe Pesci), and answers the question: How reliable is this relationship? Consider these concepts associated with the project:

- Confidence: The fraction of films containing actor A (I) that also contain actor B (j). $Confidence(I \rightarrow j) = \frac{\text{support}(I \cup \{j\})}{\text{support}(I)}$
- Lift (Interest): How much more likely B is to appear given that A is present, compared to B's overall popularity.

Table 4. Top 4 highest rules with $c = 0.5$, Lift at $s = 0.002$

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift
4	(anatoliy solonitsyn)	(nikolay grinko)	0.002	0.002	0.002	1.0	500.0
5	(nikolay grinko)	(anatoliy solonitsyn)	0.002	0.002	0.002	1.0	500.0
6	(milhem cortaz)	(andré ramiro)	0.002	0.002	0.002	1.0	500.0
7	(andré ramiro)	(milhem cortaz)	0.002	0.002	0.002	1.0	500.0

The first three rules show a Confidence of 1.0 and a very high Lift of 500. Anatoliy Solonitsyn -> Nikolay Grinko: This rule has 100% confidence. This means that in every single film in this dataset where Anatoliy Solonitsyn (I) appears, Nikolay Grinko (i) is also present. These rules have a Lift of 500: A high lift (interest) indicates that these actors are not just popular individually, but that their presence is heavily "caused" by or correlated with the other.

Table 5. Top 4 lowest rules with $c = 0.5$, Lift at $s = 0.002$

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift
119	(joe russo)	(scarlett johansson)	0.004	0.009	0.002	0.500000	55.555556
95	(franka potente)	(matt damon)	0.004	0.011	0.002	0.500000	45.454545
116	(joe pesci)	(robert de niro)	0.006	0.017	0.004	0.666667	39.215686
2	(diane keaton)	(al pacino)	0.006	0.013	0.003	0.500000	38.461538

Joe Pesci -> Robert De Niro: This rule has a Confidence of 0.666667. This implies that if Joe Pesci is in the movie, there is a 66.66 chance that De Niro is also in it. Joe Russo -> Scarlett Johansson: This rule has a Confidence of 0.50. This means that in half of the movies where Joe Russo appears, Scarlett Johansson is also a "Star" in the basket. Notice that the lift here is lower (around 39.2 to 55.5) compared to the 500 at the top list. This is because De Niro and Scarlett Johansson are very frequent items on their own. Since they appear in many movies, the surprise of finding them with a specific partner is mathematically lower than with less frequent actors.

3.2. Alternative models

Although A-priori is effective for discovering frequent item sets, it requires multiple passes over the dataset and can become computationally expensive as the number of items increases or when the s is the minimum. Those limitations motivate the use of more scalable algorithms such as SON and Toivonen's Algorithm.

3.2.1. SON algorithm

Instead of processing the entire dataset at once, SON (Savasere–Omiecinski–Navathe) algorithm divides the data into smaller chunks and runs a frequent itemset algorithm on each partition using a reduced support threshold. As described in the literature, *“the idea is to divide the input file into chunks... and run the algorithm on that chunk.”* (Jure Leskovec, 2014)

“Once all the chunks have been processed in that way, take the union of all the itemsets that have been found frequent for one or more chunks. These are the candidate itemsets.” (Jure Leskovec, 2014)

“Every itemset that is frequent in the whole is frequent in at least one chunk, and we can be sure that all the truly frequent itemsets are among the candidates; i.e., there are no false negatives. We have made a total of one pass through the data as we read each chunk and processed it. In a second pass, we count all the candidate itemsets and select those that have support at least s as the frequent itemsets.” (Jure Leskovec, 2014)

Like A-priori, at support 0.002, SON has 791 item sets, but in this case, there's is no memory crash when the threshold is 0.001, with the following result:

Table 6. Item sets at $s=0.001$ with SON

	itemsets	support
4642	(robert de niro, albert brooks)	0.001
4643	(karl hardman)	0.001
4644	(gloria stuart, claudette rains, henry travers)	0.001
4645	(jesse borrego, damian chapa)	0.001
4633	(filip berg, rolf lassgård, bahar pars)	0.001

A-priori fails at $s=0.001$ because it generates all possible candidate item sets globally, causing a combinatorial explosion in memory usage. In contrast, SON remains efficient because it partitions the dataset and performs local mining first, drastically

reducing the number of candidates item sets that must be evaluated globally. This proves that SON is more scalable than A-priori for very low support values.

The number of frequent item sets at that support equals the number of unique actor combinations that appear in at least one film. Since each film contains four actors, the theoretical maximum is 15 combinations per movie ($2^4 - 1$), leading to an upper bound of 15,000 item sets (1000×15). The observed value of 13,492 is close to this bound, indicating that most actor combinations occur only once and that overlap between combinations across films is limited.

3.2.2. Toivonen's algorithm

Toivonen's algorithm first selects a random sample and runs a frequent itemset algorithm on that sample using a slightly lower support threshold than the proportional value (e.g: 0.8ps instead of ps). As described in the lecture, *"it is essential the threshold be set to something less than its proportional value."* (Jure Leskovec, 2014) This adjustment reduces the risk of missing globally frequent itemsets. *"Having constructed the collection of frequent itemsets for the sample, we next construct the negative border. This is the collection of itemsets that are not frequent in the sample, but all of their immediate subsets (subsets constructed by deleting exactly one item) are frequent in the sample".* (Jure Leskovec, 2014)

"To complete Toivonen's algorithm, we make a pass through the entire data set, counting all the itemsets that are frequent in the sample or are in the negative border. There are two possible outcomes.

- 1. No member of the negative border is frequent in the whole dataset. In this case, the correct set of frequent itemsets is exactly those itemsets from the sample that were found to be frequent in the whole.*
- 2. Some member of the negative border is frequent in the whole. Then we cannot be sure that there are not some even larger sets, in neither the negative border nor the collection of frequent itemsets for the sample, that are*

also frequent in the whole. Thus, we can give no answer at this time and must repeat the algorithm with a new random sample.” (Jure Leskovec, 2014)

At $s = 0.002$, the shape is different than A-priori and SON, because candidate item sets are generated from a random subset of the data, and different executions may produce slightly different results.

Table 7. Item sets at $s = 0.001$ with Toivonen

	itemsets	support
3162	(zac mattoon o'brien, david thewlis)	0.001
3161	(rupert friend, david thewlis)	0.001
3160	(lesley sharp, david thewlis)	0.001
3159	(david thewlis, katrin cartlidge)	0.001
3173	(debbie reynolds, gene kelly)	0.001

At $s = 0.001$, the algorithm successfully executes without memory issues because it does not attempt to enumerate all possible item sets; instead, it first extracts candidate item sets from a random subset of the dataset and only then validates those candidates against the full data.

However, the number of frequent item sets obtained is considerably lower than the 13,492 item sets found by SON. This difference occurs because Toivonen only evaluates item sets that appear in the initial random sample. If a frequent itemset does not appear in the sample, it will never be considered as a candidate, leading to false negatives.

4. Experiments

While Toivonen improves scalability and avoids memory crashes, it sacrifices completeness in exchange for efficiency. This demonstrates the trade-off between scalability and exactness in frequent pattern mining. A-priori and SON guarantee full discovery of frequent item sets. To have a verification, the following section presents three comparative experiments conducted on the three algorithms.

4.1. Different supports

Table 8. Time measurements in seconds at $s = 0.01, 0.005$ and 0.002

	support	apriori	SON	Toivonen
0	0.010	0.008329	7.733340	0.084792
1	0.005	0.037265	9.414787	0.084927
2	0.002	0.786422	9.491508	18.448030

A-priori performs very efficiently on this dataset across the thresholds. Although execution time increases when support decreases, the absolute difference is very small. In contrast, SON exhibits high execution times regardless of the support threshold. This indicates that, for a dataset of this size, the overhead introduced by partitioning and performing two passes outweighs its scalability advantages. On the other hand, Toivonen performs efficiently at higher support values, like A-priori, but becomes significantly slower at $s = 0.002$, where candidate verification against the full dataset becomes computationally expensive. Overall, this first experiment suggests that for small to medium-sized datasets, A-priori outperform more scalable algorithms, since the additional overhead of partitioning or sampling does not provide benefits at this scale.

4.2. Different sizes

Although the experiment uses a fixed relative support threshold (0.002), the algorithms operate with absolute support values that depend on the dataset size. In the next experiment the sizes are 200, 400, 600, 800 and 1000. With the first size the support corresponds to 0.4 (≈ 1 film), with 400 it is 0.8 (≈ 1 film), with 600 it is 1.2 (≈ 2 films), with 800 it is 1.6 (≈ 2 films), and with the last corresponds to 2 films. Even with a constant relative support, the minimum number of movies required for an itemset to be considered frequent slightly increases as the dataset grows.

Table 9. Time measurements in seconds at sizes: 200, 400, 600, 800 and 1000 with $s = 0.002$

	size	Apriori	SON	Toivonen
0	200	1.055457	0.247798	0.497557
1	400	7.173269	0.936159	1.914615
2	600	0.197016	2.218771	6.035886
3	800	0.681120	4.570949	11.971242
4	1000	1.036066	9.106950	20.061256

At $s = 0.002$, SON and Toivonen exhibit a clear growth trend in runtime as dataset size increases, reflecting their dependence on scanning more baskets and validating candidates globally. Toivonen is the worst because it must both sample and verify candidates against the full dataset. Since the execution times are measured in seconds, SON and Toivonen remain significantly slower than A-priori across most tested sizes, with a substantial difference.

Table 10. Time measurements in seconds at sizes: 200, 400, 600, 800 and 1000 with $s = 0.006$

	size	Apriori	SON	Toivonen
0	200	0.012107	0.166749	0.247150
1	400	0.008656	0.549054	0.030381
2	600	0.008327	1.371999	0.062274
3	800	0.007759	2.613436	0.053349
4	1000	0.010961	4.954397	0.073918

At $s = 0.006$, A-priori remains the fastest algorithm across all dataset sizes. In contrast, SON exhibits a clear growth in runtime as the dataset size increases, reflecting the overhead introduced by partitioning the dataset and performing multiple counting phases. Toivonen is much faster than SON but still slower than A-priori. It is also important to note that SON and Toivonen internally operate with even lower effective support thresholds, because SON evaluates candidates within partitions and Toivonen generates candidates from a sampled subset before validating them on the full dataset. So, they may temporarily consider more potential item sets than A-priori, which helps them discover patterns at lower supports but also introduces additional computational overhead.

5. Scalability

Larger datasets were generated by replicating the original transaction dataset multiple times. Although this introduces duplicated transactions, it preserves the support distribution of item sets and allows scalability experiments by increasing the number of transactions.

5.1. First scalability

Table 11. Time measurements in seconds at size 1000, 2500 with $s=0.002$

	size	apriori	SON	Toivonen
0	1000	0.988065	8.470640	21.863573
1	2500	2.665438	81.039432	3.918796

Increasing the dataset size from 1,000 to 2,500 transactions has different effects across methods. A-priori does not demonstrate a clear increase. SON exhibits a huge increase, indicating that the overhead introduced by partitioning and performing multiple passes becomes increasingly expensive as data size increases. Toivonen's performance improves when the dataset size increases. This suggests that, under replicated data conditions, the sampling phase may generate a more stable or reduced candidate set, lowering the cost of the global verification step.

5.2. Second scalability

Table 12. Time measurements in seconds at size 20000, 40000 with $s = 0.01$

	size	apriori	SON	Toivonen
0	20000	0.064436	26.851750	1.282343
1	40000	0.178295	71.208765	2.459713

When increasing the dataset size from 20,000 to 40,000 transactions with a higher support threshold (0.01), all algorithms exhibit growth in execution time, confirming scalability effects. A-priori scales very efficiently under this higher support value, showing only a small increase as the dataset doubles. This indicates that when the support threshold is relatively high, the number of frequent item sets remains limited, preventing candidate explosion and allowing A-priori to remain computationally efficient even at large sizes.

In contrast, SON shows a huge growth trend as dataset size increases, suggesting that its partitioning and multiple-pass structure introduces considerable overhead at larger scales. Toivonen also increases with dataset size, but its growth is more moderate compared to SON. This result demonstrates that with higher support thresholds, classical A-priori can outperform more advanced scalable algorithms.

5.3. Third scalability

Table 13. Time measurements in seconds for Toivonen at size 10000, 20000 with $s = 0.002$

	size	Toivonen
0	10000	8.848312
1	20000	26.291265

In the last measure, only Toivonen was able to complete execution when the dataset sizes are 10,000 and 20,000 with $s = 0.002$. Although execution time increased as the dataset doubled, the algorithm remained stable and did not crash. This confirms that

the sampling-based strategy of Toivonen effectively controls candidate explosion, making it more resilient in high-volume, low-support scenarios where A-priori and SON fail due to excessive memory consumption.

6. Conclusion

Classical A-priori algorithm successfully extracted frequent actor combinations and meaningful association rules from the dataset. The results demonstrated both the practical applicability of frequent pattern mining and the importance of parameter tuning, specially, the support threshold, for obtaining informative results.

Across the experiments and scalability, the algorithm's performance strongly depends on both dataset size and support threshold. When support is relatively high, A-priori scales efficiently even as the dataset grows, since the number of frequent item sets remains limited. However, at lower support values and larger dataset sizes, memory usage becomes a critical factor: A-priori and SON may fail due to candidate explosion and RAM exhaustion. SON also exhibits significant overhead as size increases, particularly because of its partitioning and multiple-pass structure. On the other hand, Toivonen demonstrates greater robustness under large-scale, low-support conditions, as its sampling strategy helps control candidate growth and prevents memory crashes.

Overall, while A-priori may perform very well on moderate datasets, Toivonen offers better stability when scalability and memory constraints become dominant factors, but SON was the poorest, mainly because its partitioning process introduces extra overhead when executed on a single machine instead of a distributed system.

“I/We declare that this material, which I/We now submit for assessment, is entirely my/our own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my/our work, and including any code produced using generative AI systems. I/We understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me/us or any other person for assessment on this or any other course of study.”

7. References

Jure Leskovec, A. R. (2014). *Mining of Massive Datasets*. California. Retrieved from <http://www.mmids.org/>

Shankhdhar, H. (n.d.). *Kaggle*. Retrieved from Kaggle:
<https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows>